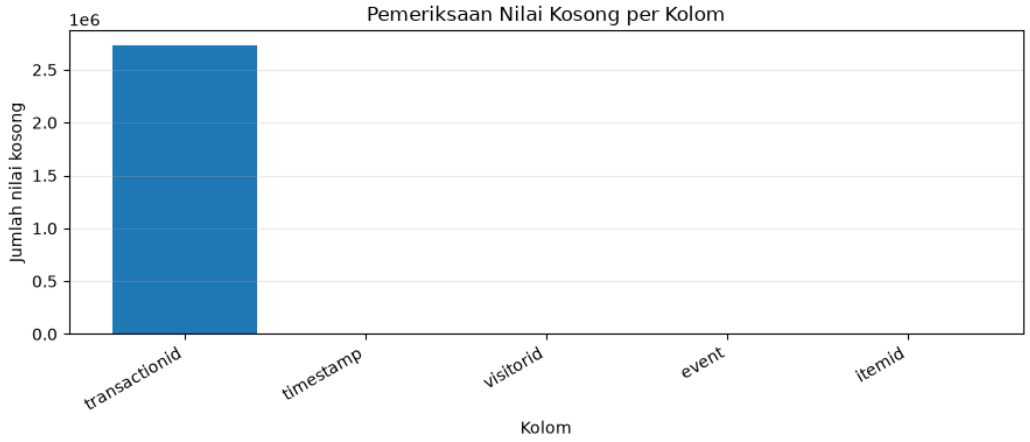
JAWABAN DAN ANALISIS UTS

Puguh Setya Wibowo
23031554088

Gambaran Data dan Persiapan

Dataset RetailRocket berisi catatan aktivitas pengunjung terhadap item. Kolom yang dipakai pada notebook adalah timestamp, visitorid, event, itemid, dan transactionid. Timestamp berbentuk Unix epoch dalam milidetik. Pemeriksaan awal menemukan 2.756.101 baris, dengan rentang waktu keseluruhan 3 Mei 2015 sampai 18 September 2015 (UTC). Tidak ada timestamp kosong atau nonnumerik. Kolom transactionid memiliki 2.733.644 nilai kosong, yang terutama terjadi pada event view dan addtocart karena event tersebut bukan transaksi pembelian.

Jenis event	Jumlah pada data penuh	Makna
view	2.664.312	Pengunjung melihat item.
addtocart	69.332	Pengunjung memasukkan item ke keranjang.
transaction	22.457	Pengunjung melakukan transaksi.
Total	2.756.101	Seluruh catatan interaksi.



Gambar 1. Pemeriksaan nilai kosong pada kolom dataset.

1. Menentukan Periode Training dan Testing

Periode yang digunakan dalam notebook adalah training 11 Mei 2015 sampai 30 Juli 2015, sedangkan testing 31 Juli 2015 sampai 20 Agustus 2015. Kedua rentang tidak tumpang tindih. Dalam kode, batas akhir dibuat eksklusif, sehingga training memakai timestamp mulai 11 Mei 2015 pukul 00.00 UTC hingga sebelum 31 Juli 2015 pukul 00.00 UTC. Testing dimulai tepat pada batas 31 Juli dan berakhir sebelum 21 Agustus 2015.

Bagian data	Tanggal awal inklusif	Tanggal akhir eksklusif	Jumlah baris
Training	11/05/2015 00.00 UTC	31/07/2015 00.00 UTC	1.732.629

Testing	31/07/2015 00.00 UTC	21/08/2015 00.00 UTC	379.855

Rentang tersebut mengikuti pembagian yang dipakai dalam notebook. Karena soal menyatakan kombinasi periode setiap mahasiswa harus berbeda, periode ini tetap perlu dipastikan sesuai dengan persetujuan atau ketentuan dosen.

2. Menerapkan Filter Timestamp dan Menyimpan CSV

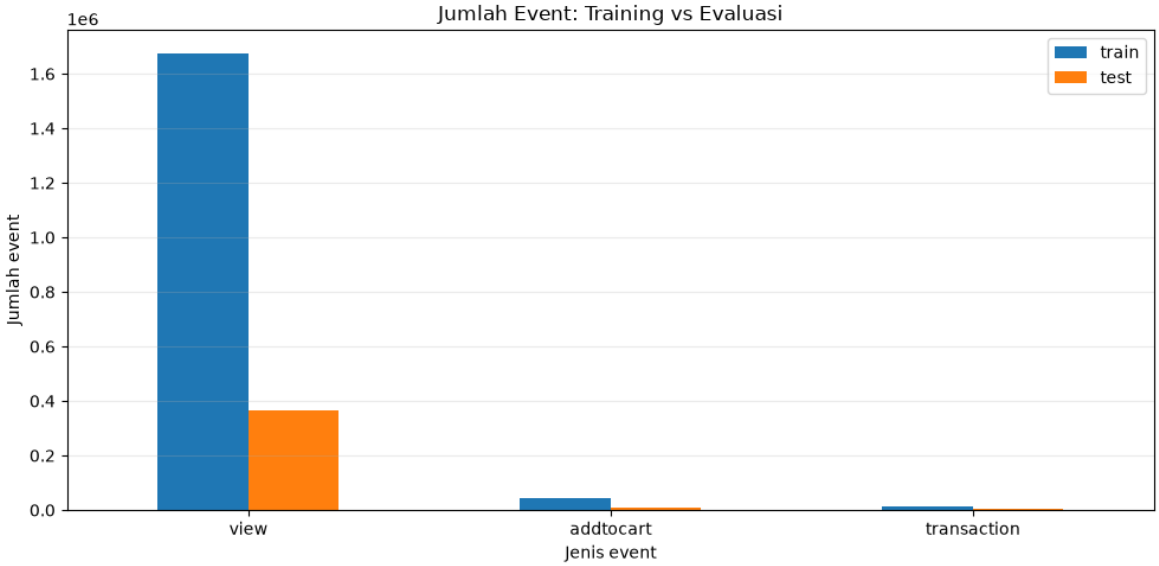
Notebook mengubah tanggal batas menjadi timestamp milidetik UTC, lalu membaca events.csv per potongan 200.000 baris agar pemrosesan tidak harus memuat seluruh dataset sekaligus. Setiap potongan dipisahkan memakai kondisi timestamp. Baris yang masuk ke periode training ditulis ke events_train.csv, sedangkan baris yang masuk ke periode testing ditulis ke events_test.csv.

```
TR_START, TR_END = ms("2015-05-11"), ms("2015-07-31")
TE_START, TE_END = ms("2015-07-31"), ms("2015-08-21")

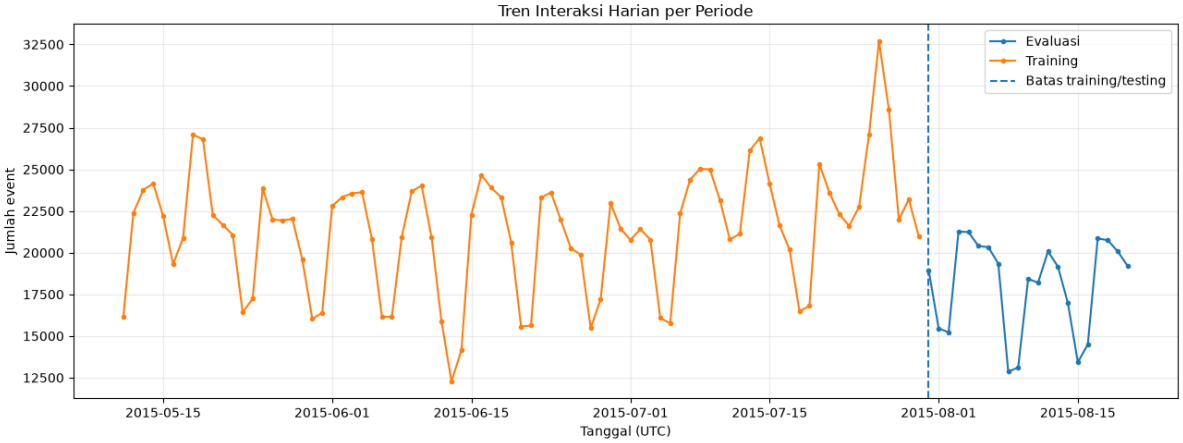
train_mask = (timestamp >= TR_START) & (timestamp < TR_END)
test_mask  = (timestamp >= TE_START) & (timestamp < TE_END)
```

Jenis event	Training	Testing
view	1.676.031	366.500
addtocart	42.494	10.183
transaction	14.104	3.172
Total	1.732.629	379.855

Hasil filter menunjukkan bahwa event view paling banyak pada kedua periode. Jumlah event testing lebih kecil karena rentang waktunya lebih pendek. Kedua CSV ini kemudian dipakai untuk analisis berikutnya. Pemisahan waktu juga mencegah interaksi dari periode testing ikut membentuk VSM dan similarity pada tahap training.



Gambar 2. Perbandingan jumlah event training dan testing.



Gambar 3. Tren interaksi harian pada dua periode. Garis putus-putus menandai batas training dan testing.

3. Komposisi Subgraph Training dengan Python API

Setelah data tersedia di Neo4j, notebook menjalankan query melalui Neo4j Python Driver. Subgraph dibatasi dengan timestamp training, lalu dihitung jumlah Visitor unik, Item unik, dan seluruh relasi interaksi yang berada pada rentang waktu tersebut.

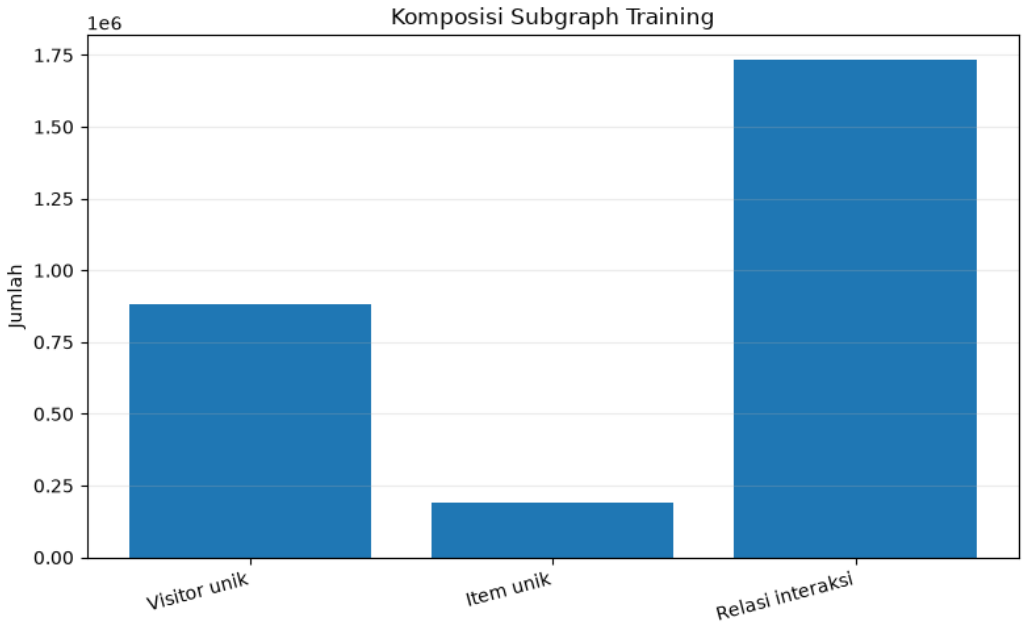
```
MATCH (v:Visitor)-[r]->(i:Item)
WHERE r.timestamp >= $start AND r.timestamp < $end
RETURN count(DISTINCT v) AS visitors,
       count(DISTINCT i) AS items,
       count(r) AS relationships
```

Penjelasan query: MATCH mencari pola Visitor yang terhubung ke Item melalui relasi apa pun. WHERE membatasi relasi pada periode training. count(DISTINCT v) menghitung setiap pengunjung hanya satu kali, count(DISTINCT i) menghitung item unik, sedangkan count(r) menghitung setiap kejadian relasi. Parameter $start dan $end diisi dari batas timestamp yang dihitung di Python.

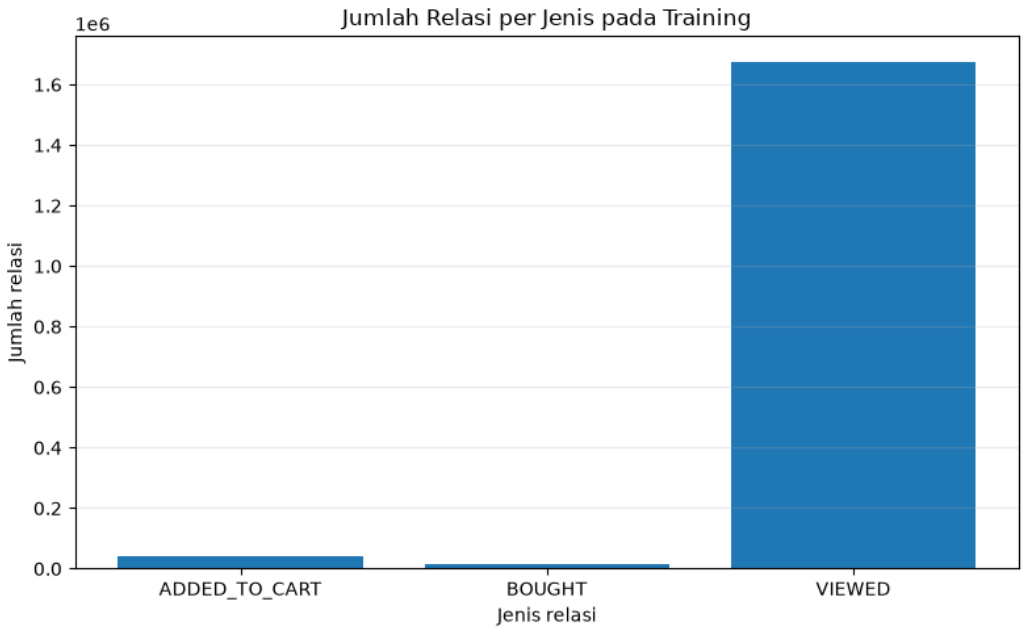
Komponen	Jumlah pada subgraph training	Penjelasan
Simpul Visitor	882.707	Pengunjung unik yang memiliki interaksi selama training.
Simpul Item	192.332	Item unik yang berinteraksi dengan pengunjung selama training.
Relasi interaksi	1.732.629	Semua kejadian interaksi dalam periode training.

Jenis relasi	Jumlah	Arti
VIEWED	1.676.031	Melihat item.
ADDED_TO_CART	42.494	Menambahkan item ke keranjang.
BOUGHT	14.104	Membeli item.

Jumlah relasi sama dengan total baris training, yaitu 1.732.629. Relasi VIEWED mendominasi data. Hal ini berarti aktivitas paling umum adalah melihat produk, sedangkan transaksi pembelian jauh lebih sedikit. Perbedaan jumlah tersebut perlu diperhatikan ketika membangun model karena interaksi yang lebih sering tidak selalu berarti minat beli yang sama kuat.



Gambar 4. Jumlah Visitor, Item, dan relasi pada subgraph training.



Gambar 5. Jumlah relasi menurut jenis event.

4. VSM pada Simpul User

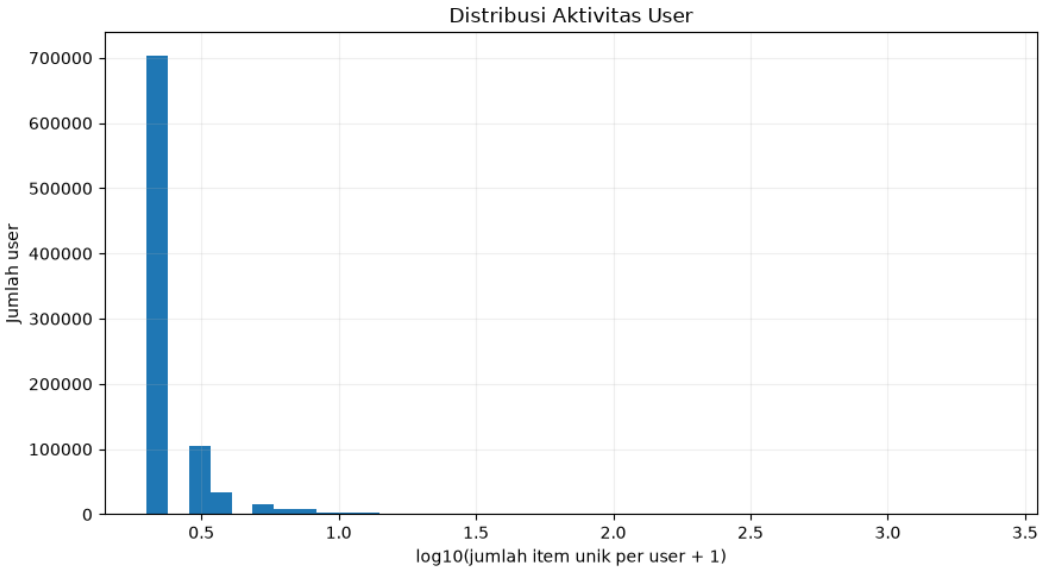
Vector Space Modelling (VSM) mengubah aktivitas pengunjung menjadi matriks numerik. Pada VSM user-item, setiap baris mewakili satu Visitor dan setiap kolom mewakili satu Item. Isi matriks adalah bobot interaksi pengunjung terhadap item. Notebook memberi bobot view = 1, addtocart = 2, dan transaction = 3. Dengan cara ini, memasukkan item ke keranjang diberi bobot lebih tinggi daripada sekadar melihat item, sedangkan transaksi diberi bobot tertinggi.

```
df["weight"] = df["event"].map({
    "view": 1.0,
    "addtocart": 2.0,
    "transaction": 3.0
})
X = coo_matrix((df.weight, (user_index, item_index)),
               shape=(jumlah_user, jumlah_item)).tocsr()
X.sum_duplicates()
```

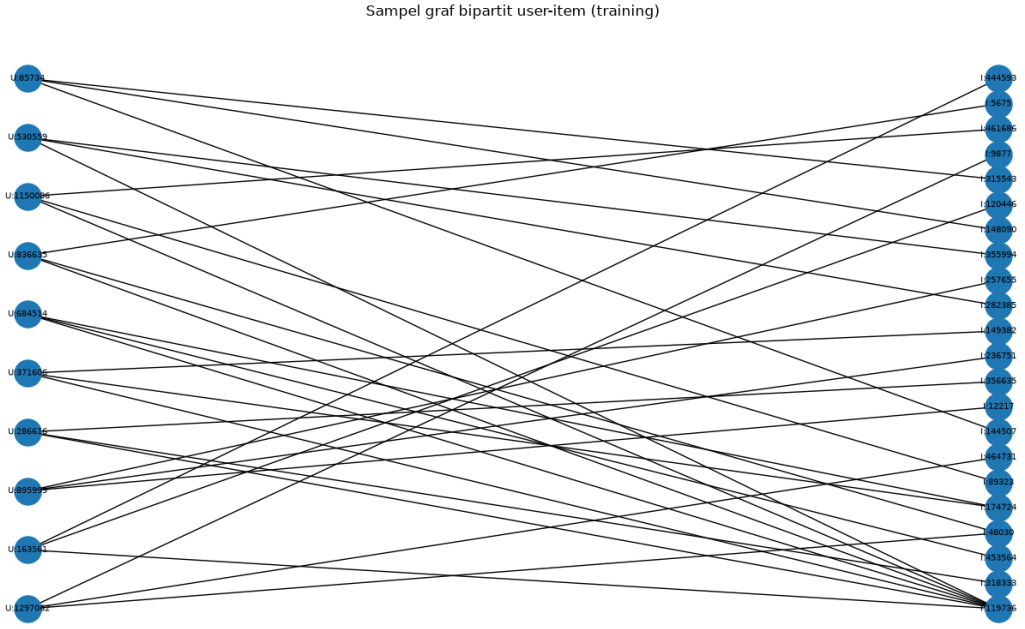
Fungsi factorize mengubah ID Visitor dan Item menjadi indeks baris dan kolom. Matriks dibuat dalam format sparse CSR agar pasangan user-item yang tidak memiliki interaksi tidak perlu disimpan sebagai nilai nol.

Karakteristik VSM User	Hasil
Shape matriks	(882.707, 192.332)
Jumlah pasangan nonnol	1.345.274
Kemungkinan pasangan user-item	169.772.802.724
Densitas	0,00079240%
Sparsity	99,99920760%
Rata-rata item unik per user	1,524
Median item unik per user	1
Maksimum item unik per user	2.460

Sparsity yang sangat tinggi menunjukkan sebagian besar pengunjung hanya berinteraksi dengan sebagian kecil dari seluruh item. Rata-rata jumlah item unik per user hanya sekitar 1,52 dan mediannya 1. Ada pula sejumlah kecil user yang berinteraksi dengan banyak item, hingga 2.460 item pada data training.



Gambar 6. Distribusi jumlah item unik per user, ditampilkan dalam skala logaritmik.



Gambar 7. Sampel graf bipartit user-item dari training. Graf menampilkan 10 user teraktif dan hingga tiga item per user, bukan seluruh graf.

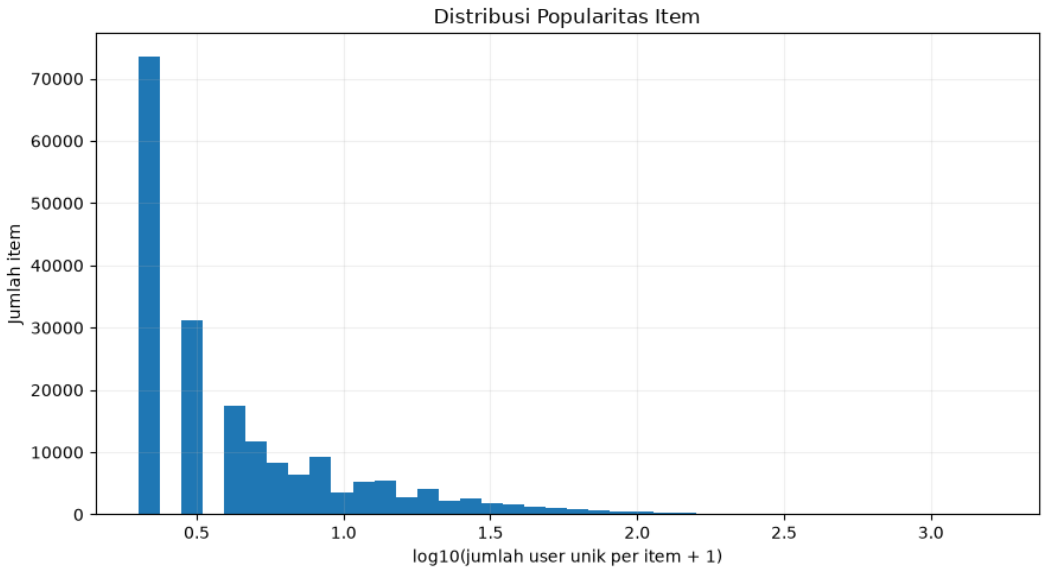
5. VSM pada Simpul Item

Untuk melihat data dari sisi item, matriks user-item ditransposisikan menjadi matriks item-user. Pada matriks ini, setiap baris adalah Item dan setiap kolom adalah Visitor. Dengan demikian, vektor item berisi bobot interaksi dari para pengunjung yang berinteraksi dengannya. Representasi ini berguna untuk membandingkan item berdasarkan pola pengunjung yang sama.

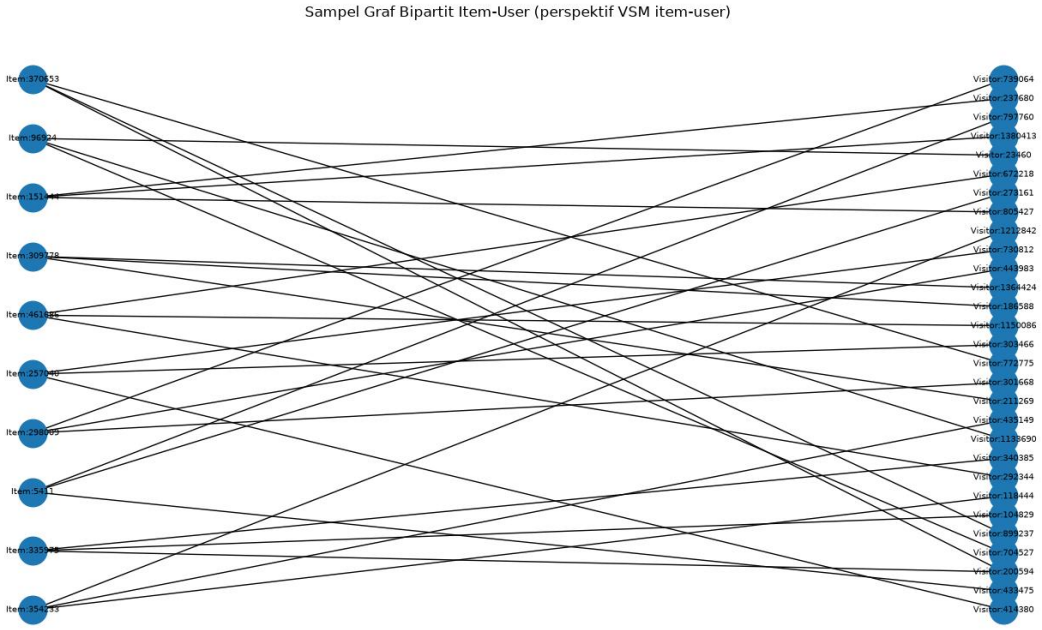
```
X_item_user = X.T.tocsr()
```

Karakteristik VSM Item	Hasil
Shape matriks	(192.332, 882.707)
Rata-rata user unik per item	6,995
Median user unik per item	2
Maksimum user unik per item	1.669

Matriks item-user memiliki dimensi terbalik dari user-item, tetapi informasi interaksinya sama. Rata-rata item diinteraksikan oleh sekitar tujuh user, sedangkan median hanya dua user. Ini menunjukkan bahwa sebagian besar item memiliki jangkauan interaksi kecil, sementara beberapa item memiliki popularitas jauh lebih tinggi.



Gambar 8. Distribusi jumlah user unik per item.



Gambar 9. Sampel graf bipartit dengan item sebagai sisi utama. Graf menampilkan 10 item teraktif dan hingga tiga pengunjung per item.

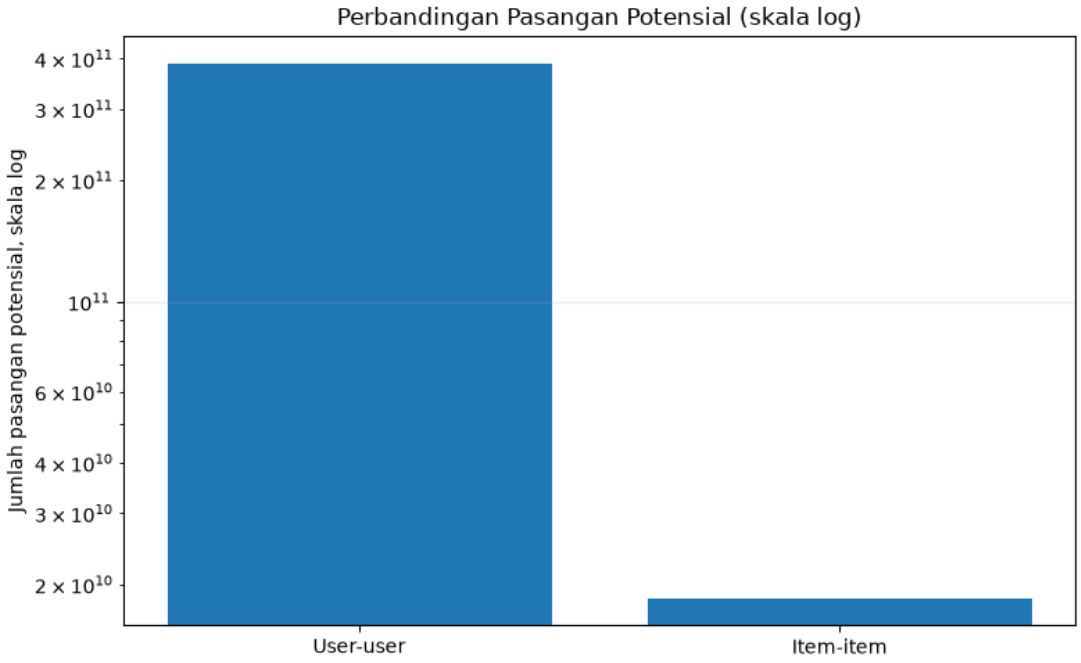
6. Menentukan Proyeksi yang Lebih Efektif dan Efisien

Proyeksi user-user membandingkan kemiripan antar-pengunjung berdasarkan item yang mereka interaksikan. Proyeksi item-item membandingkan kemiripan antar-item berdasarkan pengunjung yang berinteraksi dengannya. Karena tujuan lanjutan notebook adalah mencari item yang mirip untuk rekomendasi, proyeksi item lebih sesuai dengan tujuan analisis.

Jenis proyeksi	Jumlah node	Pasangan potensial
User–User	882.707	389.585.382.571
Item–Item	192.332	18.495.702.946

Jumlah pasangan potensial dihitung dengan n(n−1)/2. Proyeksi user memiliki sekitar 389,6 miliar pasangan, sedangkan proyeksi item sekitar 18,5 miliar. Dengan kata lain, proyeksi user memiliki sekitar 21,06 kali lebih banyak pasangan potensial. Proyeksi item karena itu lebih hemat sebagai arah pencarian similarity dan lebih relevan dengan pertanyaan rekomendasi produk.

Walaupun lebih kecil, 18,5 miliar pasangan item tetap sangat besar jika seluruhnya dihitung sekaligus. Notebook tidak membentuk seluruh proyeksi item secara padat. Sebaliknya, notebook menghitung similarity untuk satu item acuan dengan operasi matriks sparse, sehingga kebutuhan memori lebih terkendali.



Gambar 10. Perbandingan pasangan potensial proyeksi user dan item dengan sumbu logaritmik.

7. Menghitung Cosine Similarity dan Menyimpan Nilainya ke Neo4j

Pemilihan item acuan dan hasil Python

Notebook memilih item acuan menggunakan seed 42 dari item yang telah berinteraksi dengan minimal 10 user pada periode training. Item acuan yang terpilih adalah 42874 dan memiliki 21 user. Perhitungan cosine similarity menghasilkan 745 kandidat item dengan hasil kali vektor positif.

Cosine similarity mengukur kesearahan dua vektor interaksi. Nilai yang mendekati 1 menunjukkan pola interaksi yang lebih mirip, sedangkan nilai yang mendekati 0 menunjukkan kemiripan yang lebih rendah.

```
cosine(A, B) = (A · B) / (||A|| × ||B||)
```

Item acuan	Item mirip dari hasil Python	Similarity
42874	317192	0,775165
42874	16370	0,709550
42874	28161	0,680847
42874	275580	0,515896
42874	133250	0,450988

Item 317192 memperoleh skor tertinggi, yaitu 0,775165. Berdasarkan representasi interaksi pada training, pola user untuk item tersebut paling searah dengan pola user item 42874 di antara kandidat yang dihitung.

7(a). Query Cypher untuk mengambil Top 5

```
MATCH (b:Item {itemId: 42874})-[r:SIMILAR_TO]->(c:Item)
RETURN b.itemId AS item_acuan,
      c.itemId AS item_mirip,
      r.similarity AS similarity
ORDER BY similarity DESC
LIMIT 5;
```

Query mencari relasi SIMILAR_TO yang berawal dari item 42874, mengambil ID item acuan, ID item kandidat, dan properti similarity, kemudian mengurutkan skor dari yang tertinggi dan membatasi hasil menjadi lima baris.

7(b). Menyimpan nilai similarity ke graf dan memeriksa hasil query

Notebook menyimpan hasil similarity sebagai relasi terarah SIMILAR_TO. Setiap relasi memiliki properti similarity, method = 'cosine', dan data_split = 'training'. Sebelum menulis batch baru, relasi SIMILAR_TO dari item acuan yang sama dihapus, kemudian kandidat disimpan per batch agar proses penulisan lebih terkontrol.

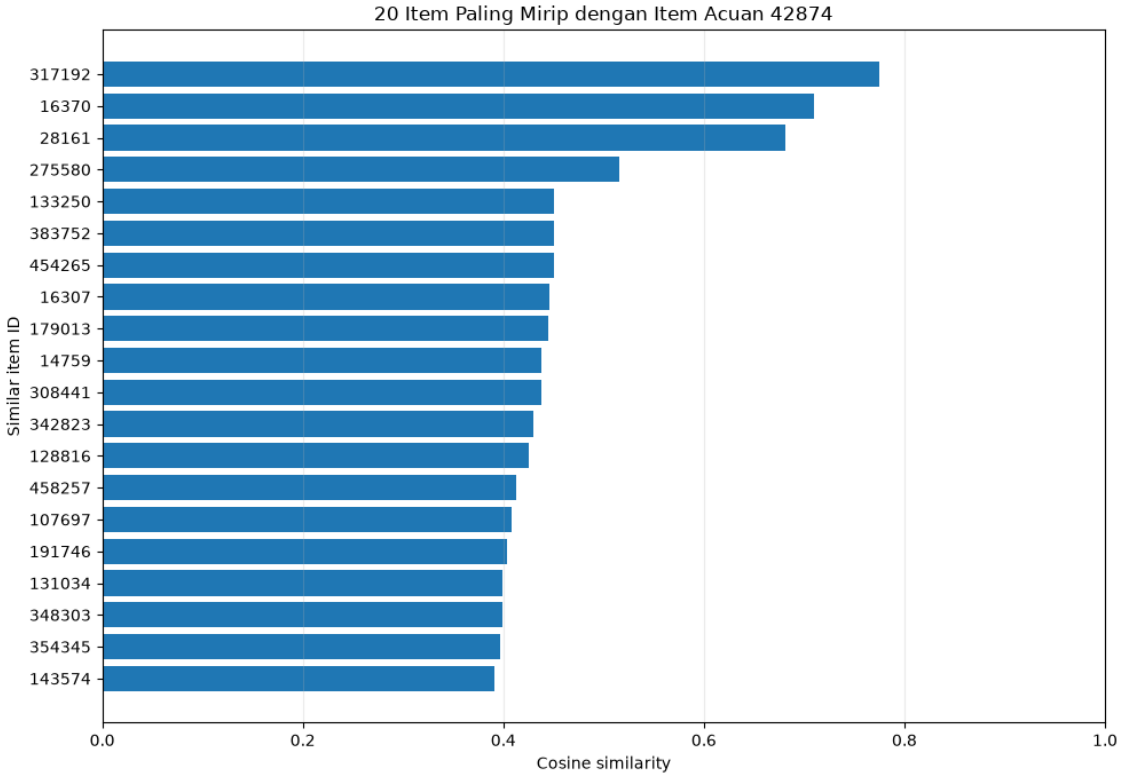
```
UNWIND $rows AS row
MERGE (b:Item {itemId: toInteger(row.reference_item_id)})
MERGE (c:Item {itemId: toInteger(row.similar_item_id)})
MERGE (b)-[r:SIMILAR_TO]->(c)
SET r.similarity = toFloat(row.similarity),
   r.method = 'cosine',
   r.data_split = 'training'
```

Table Raw

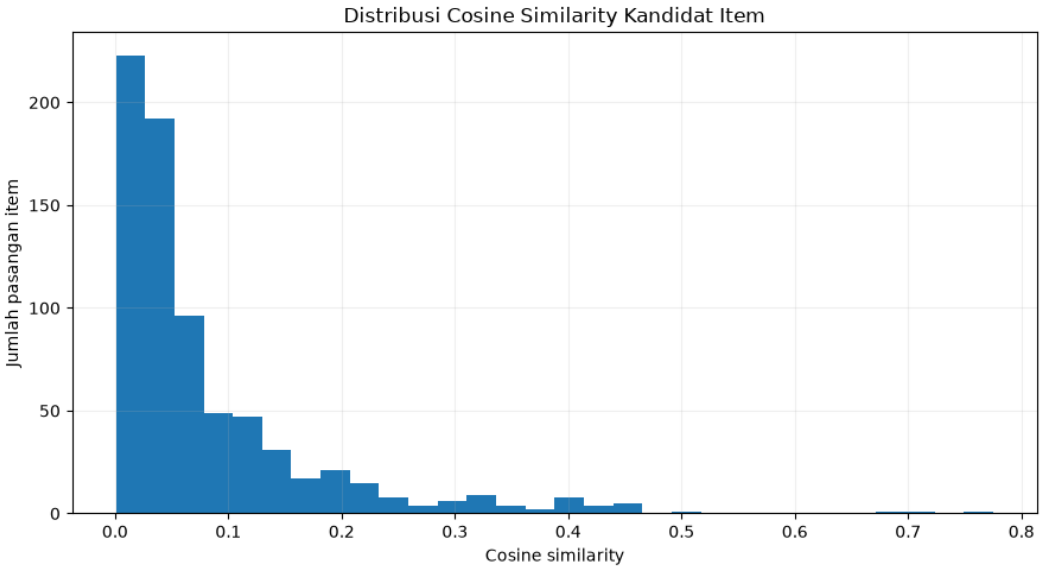
	item_acuan	item_mirip	similarity
1	42874	317192	0.7751649618148804
2	42874	16370	0.7095497846603394
3	42874	28161	0.6808468103408813
4	42874	275580	0.5158957839012146
5	42874	133250	0.45098763704299927

Started streaming 5 records after 1,119 ms and completed after 1,327 ms.

Ada perbedaan pada baris kelima antara tabel Python dan hasil query Neo4j: Python menampilkan item 133250, sedangkan Neo4j menampilkan item 454265. Keduanya memiliki similarity 0,450988. Penyebabnya adalah query notebook hanya mengurutkan berdasarkan similarity. Karena dua item memiliki skor sama, urutan baris kelima tidak dijamin sama. Untuk membuat urutan konsisten, query dapat ditulis ORDER BY r.similarity DESC, c.itemId ASC lalu dijalankan ulang.



Gambar 11. Dua puluh item dengan similarity tertinggi terhadap item 42874.



Gambar 12. Distribusi skor similarity kandidat item.

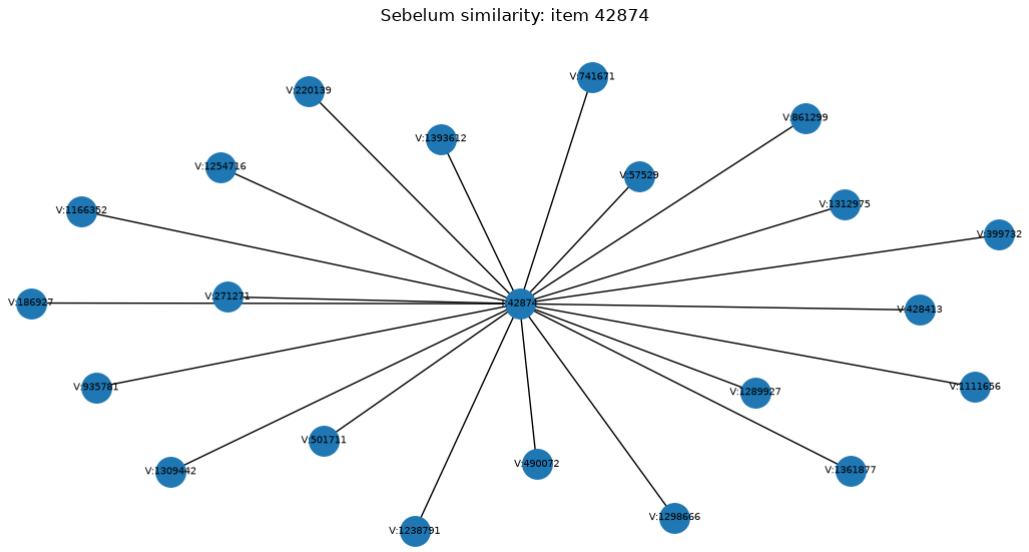
8. Kondisi Graf Sebelum dan Sesudah Similarity

Sebelum similarity: hubungan interaksi asli

Sebelum similarity ditambahkan, graf hanya memperlihatkan hubungan interaksi yang memang tercatat dalam dataset training. Pada contoh ini, item 42874 terhubung dengan pengunjung yang melakukan aktivitas terhadap item tersebut. Jenis aktivitas dapat berupa VIEWED, ADDED_TO_CART, atau BOUGHT.

```
MATCH (v:Visitor)-[r]->(i:Item {itemId: 42874})
WHERE r.timestamp >= $start AND r.timestamp < $end
RETURN v, r, i
LIMIT 30;
```

Pada notebook, hasil query dibaca melalui Python Driver lalu disusun menjadi graf NetworkX. Query pada Neo4j Browser yang mengembalikan v, r, dan i dapat digunakan untuk menampilkan simpul serta relasi langsung pada tampilan Graph.



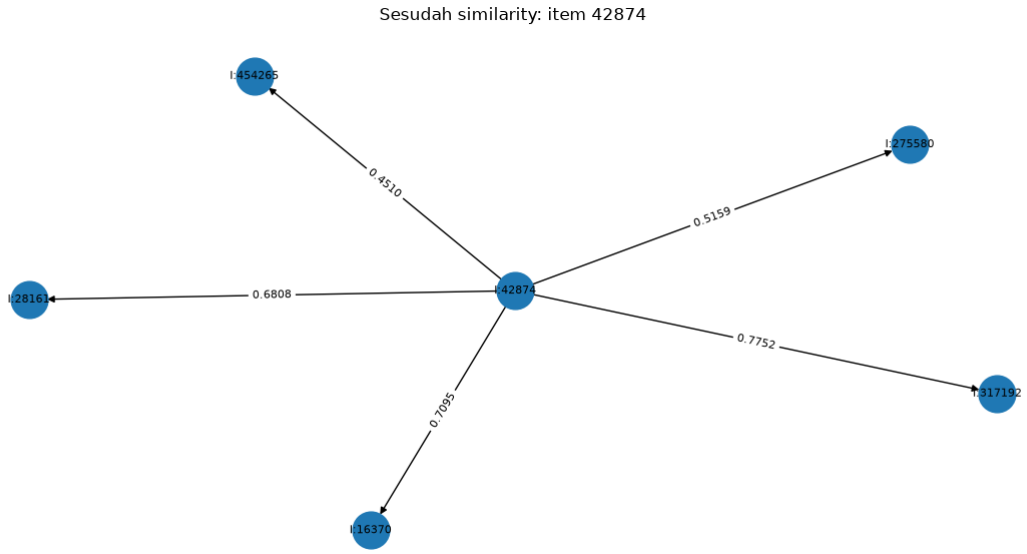
Gambar 13. Graf sebelum similarity, berdasarkan relasi interaksi training untuk item 42874.

Sesudah similarity: hubungan antar-item

Setelah cosine similarity dihitung dan disimpan, graf menampilkan hubungan item 42874 dengan item-item yang mempunyai pola interaksi mirip. Relasi SIMILAR_TO membawa nilai similarity sebagai properti, sehingga skor dapat dibaca dari query dan ditampilkan pada label relasi.

```
MATCH (b:Item {itemId: 42874})-[r:SIMILAR_TO]->(c:Item)
RETURN b, r, c
ORDER BY r.similarity DESC
LIMIT 5;
```

Pada tahap ini, graf tidak lagi hanya menunjukkan siapa yang melihat atau membeli item acuan. Graf juga memperlihatkan item mana yang direkomendasikan sebagai item serupa berdasarkan pola interaksi. Notebook membuat visualisasi NetworkX dari lima record yang dikembalikan oleh Neo4j. Jadi, graf pada dokumen merupakan visualisasi dari hasil query database; jika dosen meminta bukti Neo4j Browser, screenshot tampilan Graph perlu diambil secara terpisah.



Gambar 14. Graf sesudah similarity, yaitu item 42874 terhubung ke item serupa melalui relasi SIMILAR_TO.

Kesimpulan Bagian Wrangling dan Similarity Item

Data difilter menjadi 1.732.629 baris training dan 379.855 baris testing. Subgraph training terdiri atas 882.707 Visitor, 192.332 Item, dan 1.732.629 relasi. VSM user-item mempunyai shape 882.707 × 192.332, sedangkan VSM item-user mempunyai shape 192.332 × 882.707. Sparsity matriks mencapai 99,99920760%, sehingga format sparse lebih tepat daripada menyimpan seluruh pasangan sebagai matriks padat.

Dari sisi efisiensi, proyeksi item memiliki sekitar 21,06 kali lebih sedikit pasangan potensial daripada proyeksi user dan lebih sesuai untuk tujuan mencari item yang mirip. Item acuan 42874 menghasilkan kandidat similarity tertinggi 317192 dengan skor 0,775165. Nilai similarity disimpan ke Neo4j sebagai relasi SIMILAR_TO, kemudian diperiksa kembali melalui query Cypher. Perbedaan item peringkat kelima pada Python dan Neo4j terjadi karena skor similarity seri dan query belum menggunakan aturan pengurutan kedua.

Seluruh jawaban pada laporan ini menggunakan nilai dan visualisasi yang tersimpan pada notebook yang diunggah. Satu hal yang masih perlu diperhatikan saat pengumpulan adalah memastikan periode tersebut memang disetujui dosen, karena instruksi menyebut kombinasi periode tiap mahasiswa tidak boleh sama.

9. Soal 1: Rekomendasi Item Menggunakan KNN

Metode yang digunakan adalah User-Based Collaborative Filtering dengan algoritma K-Nearest Neighbors (KNN). Sistem mencari lima pengguna yang memiliki pola interaksi paling mirip dengan pengguna target berdasarkan cosine similarity. Skor kandidat item dihitung dari penjumlahan bobot interaksi lima tetangga yang dikalikan dengan nilai similarity masing-masing. Item yang sudah pernah berinteraksi dengan pengguna target pada data training tidak dimasukkan kembali ke rekomendasi.

9.1(a) Skrip Python untuk Rekomendasi KNN

Pemilihan pengguna dibuat reproducible dengan seed = 42. Skrip berikut dijalankan setelah matriks sparse user-item X, daftar user_ids dan item_ids, serta file events_test.csv disiapkan pada tahap sebelumnya. Urutan user_ids dan item_ids harus sesuai dengan baris dan kolom matriks X.

```
import numpy as np
import pandas as pd

# X adalah matriks user-item sparse dari data training.
# user_ids dan item_ids harus sejajar dengan baris/kolom X.
TEST = "events_test.csv"
SEED_KNN = 42
K = 5
TOP_N = 5
MAX_ATTEMPTS = 200

X = X.tocsr()
X.sort_indices()

# Pemetaan ID asli ke indeks matriks, tanpa mengasumsikan ID sudah terurut.
user_to_idx = {int(uid): idx for idx, uid in enumerate(user_ids)}
item_to_idx = {int(iid): idx for idx, iid in enumerate(item_ids)}

test_df = pd.read_csv(
    TEST,
    usecols=["visitorid", "itemid", "event"]
)
test_df["event"] = test_df["event"].astype(str).str.lower().str.strip()
test_df = test_df[
    test_df["event"].isin(["view", "addtocart", "transaction"])
].copy()

test_df["ui"] = test_df["visitorid"].map(user_to_idx)
test_df["ii"] = test_df["itemid"].map(item_to_idx)
test_df = test_df.dropna(subset=["ui", "ii"]).copy()

ui = test_df["ui"].to_numpy(dtype=np.int64)
ii = test_df["ii"].to_numpy(dtype=np.int64)

# Tandai pasangan user-item yang sudah muncul pada data training.
train_rows = np.repeat(
    np.arange(X.shape[0], dtype=np.int64),
    np.diff(X.indptr)
)
train_keys = (
    train_rows * np.int64(X.shape[1])
    + X.indices.astype(np.int64)
)
test_keys = ui * np.int64(X.shape[1]) + ii

pos = np.searchsorted(train_keys, test_keys)
            inrange = pos < len(train_keys)
seen = np.zeros(len(pos), dtype=bool)
seen[inrange] = train_keys[pos[inrange]] == test_keys[inrange]
unseen = ~seen

# Kandidat user harus memiliki minimal satu item testing yang belum pernah
# muncul pada training.
eligible_users = np.unique(ui[unseen])
rng = np.random.default_rng(SEED_KNN)
user_order = rng.permutation(eligible_users)

user_norms = np.sqrt(
```

```
    np.asarray(X.multiply(X).sum(axis=1)).ravel()
)
chosen = None

for attempt, target in enumerate(user_order[:MAX_ATTEMPTS], start=1):
    target = int(target)
    vec = X.getrow(target)

    # Cosine similarity semua user terhadap user target.
    dot = X.dot(vec.T).toarray().ravel()
    denom = user_norms * user_norms[target]
    sims = np.divide(
        dot,
        denom,
        out=np.zeros_like(dot, dtype=float),
        where=denom > 0
    )
    sims[target] = -1.0

    possible = np.flatnonzero(sims > 0)
    if len(possible) < K:
        continue

    # Urutkan similarity menurun; jika seri, gunakan ID user menaik.
    user_array = np.asarray(user_ids)
    order = np.lexsort((user_array[possible], -sims[possible]))
    neighbors = possible[order[:K]]
    neighbor_sims = sims[neighbors]

    # Skor item = jumlah bobot interaksi tetangga x similarity tetangga.
    scores = np.asarray(X[neighbors].T.dot(neighbor_sims)).ravel()

    # Jangan rekomendasikan item yang sudah muncul pada training user target.
    scores[vec.indices] = -np.inf
    candidates = np.flatnonzero(np.isfinite(scores) & (scores > 0))
    if len(candidates) < TOP_N:
        continue

    ranked = candidates[
        np.lexsort((np.asarray(item_ids)[candidates], -scores[candidates]))
        [:TOP_N]
    ]
    relevant = np.unique(ii[unseen & (ui == target)])

    chosen = {
        "target": target,
        "neighbors": neighbors,
        "neighbor_sims": neighbor_sims,
        "scores": scores,
        "ranked": ranked,
        "relevant": relevant,
        "attempt": attempt
    }
    break

if chosen is None:
    raise RuntimeError("Tidak ditemukan user yang memenuhi kriteria.")

target = chosen["target"]
print("Seed:", SEED_KNN)
print("Percobaan:", chosen["attempt"])
print("User target:", int(user_ids[target]))
print("Item relevan pada testing:", item_ids[chosen["relevant"]].tolist())

neighbors_df = pd.DataFrame({
    "neighbor_user_id": np.asarray(user_ids)[chosen["neighbors"]],
    "cosine_similarity": chosen["neighbor_sims"]
})
print("\\nLima tetangga terdekat:")
print(neighbors_df.to_string(index=False))

recommendations_df = pd.DataFrame({
    "rank": np.arange(1, len(chosen["ranked"]) + 1),
    "item_id": np.asarray(item_ids)[chosen["ranked"]],
    "score": chosen["scores"][chosen["ranked"]]
})
```

```
print("\\nLima item rekomendasi:")
print(recommendations_df.to_string(index=False))

# Evaluasi Top 5 dengan relevansi biner dari data testing.
relevant_set = set(chosen["relevant"].tolist())
top5 = chosen["ranked"][:5]
rel = np.array([int(i in relevant_set) for i in top5], dtype=float)

hit5 = int(rel.sum() > 0)
dcg5 = np.sum(rel / np.log2(np.arange(2, len(rel) + 2)))
ideal_len = min(len(relevant_set), 5)
idcg5 = np.sum(1 / np.log2(np.arange(2, ideal_len + 2))) if ideal_len else 0.0
ndcg5 = dcg5 / idcg5 if idcg5 > 0 else 0.0

print("Hit@5:", hit5)
print("NDCG@5:", round(float(ndcg5), 6))
```

9.1(b) Hasil Pemilihan User dan Tiga Item Rekomendasi

Dengan seed 42, pengguna target yang terpilih pada hasil notebook adalah user 335902 pada percobaan ke-24. Item relevan yang tercatat pada data testing untuk pengguna ini adalah item 268883.

Lima tetangga terdekat untuk user target ditunjukkan pada tabel berikut.

User tetangga	Cosine similarity
1230943	0,707107
259962	0,577350
428608	0,500000
139139	0,196116
340817	0,176777

Tiga item dengan skor rekomendasi tertinggi adalah sebagai berikut.

Peringkat	ID item	Skor rekomendasi
1	69917	0,754127
2	273604	0,577350
3	323676	0,549670

Item 69917 menempati peringkat pertama karena memperoleh skor tertinggi, yaitu 0,754127. Skor yang lebih tinggi menunjukkan bahwa item tersebut mendapat dukungan interaksi lebih besar dari lima tetangga terdekat.

10. Soal 2: Evaluasi Hasil Rekomendasi

Evaluasi menggunakan lima item teratas dari daftar rekomendasi. Item relevan pada data testing adalah 268883, tetapi item tersebut tidak muncul pada Top 5 rekomendasi.

Peringkat	ID item	Skor	Relevan pada testing?
1	69917	0,754127	Tidak
2	273604	0,577350	Tidak
3	323676	0,549670	Tidak
4	404920	0,500000	Tidak
5	193249	0,353553	Tidak

10.1(a) Perhitungan Hit@5

Hit@5 bernilai 1 apabila setidaknya satu item relevan muncul dalam lima rekomendasi teratas, dan bernilai 0 apabila tidak ada item relevan yang ditemukan. Pada hasil ini, jumlah item relevan yang muncul di Top 5 adalah nol.

Hit@5 = 0

10.2(b) Perhitungan NDCG@5

NDCG@5 mengukur kualitas urutan rekomendasi dengan memberi bobot lebih besar kepada item relevan yang berada pada peringkat lebih tinggi. Rumus DCG yang digunakan adalah:

$$DCG@5 = \Sigma \, rel_i / \log_2(i + 1), \text{ untuk } i = 1 \text{ sampai } 5$$

Karena tidak ada item relevan pada Top 5, nilai rel_i pada seluruh peringkat adalah 0, sehingga DCG@5 = 0. Terdapat satu item relevan pada data testing, maka $IDCG@5 = 1 / \log_2(2) = 1$.

$$\mathbf{NDCG@5 = DCG@5 / IDCG@5 = 0 / 1 = 0}$$

Dengan demikian, Hit@5 = 0 dan NDCG@5 = 0. Sistem belum berhasil menempatkan item relevan 268883 ke dalam lima rekomendasi teratas untuk user 335902.

Kesimpulan Akhir

Pada bagian wrangling dan EDA, dataset RetailRocket dibagi menjadi 1.732.629 baris training dan 379.855 baris testing. Matriks user-item sangat sparse, dengan sparsity 99,99920760%, sehingga format sparse lebih sesuai untuk pemrosesan. Pada analisis similarity item, item acuan 42874 menghasilkan item paling mirip 317192 dengan cosine similarity 0,775165.

Pada percobaan KNN dengan seed 42, user target yang terpilih adalah 335902. Tiga rekomendasi teratas adalah item 69917, 273604, dan 323676. Namun, item relevan pada testing, yaitu 268883, tidak ditemukan pada Top 5. Oleh karena itu, hasil evaluasi menunjukkan Hit@5 = 0 dan NDCG@5 = 0. Hasil ini menunjukkan bahwa rekomendasi pada percobaan tersebut belum berhasil menemukan item relevan untuk pengguna target.